

10. Improving Structure with Inheritance

Example 10.1

```
abstract class Animal {
    public abstract void makeSound();
}

class Dog extends Animal {
    public void makeSound() {
        System.out.println("Woof");
    }

    public void fetch() {
        System.out.println("Fetching");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();    // polymorphic variable
        a.makeSound();           // subtype substitution

        Dog d = (Dog) a;        // cast
        d.fetch();
    }
}
```

Example 10.2

```
Animal[] animals = new Animal[2];
animals[0] = new Dog();
animals[1] = new Dog();

for (Animal a : animals) {
    a.makeSound();
}
```

10.1. Inheritance

Definition

Inheritance allows a class to reuse fields and methods from another class.

Features

- One class extends another class
- Code reuse

- Models real-world **is-a** relationships

Example

```
class Dog extends Animal { }
```

- Dog inherits from Animal

Remark

- Java supports **single inheritance** only
-

10.2. Superclass and Subclass

Definition

A superclass is the parent class.

A subclass is the child class that inherits from it.

Features

- Superclass provides common behavior
- Subclass adds or overrides behavior

Example

```
class Dog extends Animal { }
```

- Animal is the superclass
- Dog is the subclass

Remark

- Use `extends` keyword
-

10.3. Is-a Relationship

Definition

If class B extends class A, then B **is-a** A.

Features

- Fundamental rule of inheritance
- Enables polymorphism

Example

```
Animal a = new Dog();
```

- A Dog is an Animal

Remark

- If **is-a** is false, inheritance is wrong
-

10.4. Inheritance Hierarchy

Definition

An inheritance hierarchy is a tree of related classes.

Features

- Root class at the top
- Multiple levels of inheritance

Example

```
class Animal { }  
class Mammal extends Animal { }  
class Dog extends Mammal { }
```

Remark

- Deeper hierarchies increase complexity
-

10.5. Abstract Class

Definition

An abstract class cannot be instantiated and may contain abstract methods.

Features

- Uses `abstract` keyword
- Forces subclasses to implement methods

Example

```
abstract class Animal {  
    public abstract void makeSound();  
}
```

Remark

- Used as a **base type**
-

10.6. Subtyping

Definition

A subclass type is a subtype of its superclass type.

Features

- Based on inheritance
- Supports assignment compatibility

Example

```
Animal a = new Dog();
```

Remark

- Subtyping is a **type system rule**
-

10.7. Substitution (Subtype Substitution)

Definition

An object of a subtype can be used where a supertype is expected.

Features

- Core of polymorphism
- Method calls use runtime type

Example

```
Animal a = new Dog();  
a.makeSound();
```

Remark

- Also called **Liskov Substitution Principle (basic form)**
-

10.8. Polymorphic Variable

Definition

A polymorphic variable is a variable whose declared type is a superclass.

Features

- Holds objects of different subclasses
- Method binding is dynamic

Example

```
Animal a = new Dog();
```

Remark

- Very common in arrays and collections
-

10.8.1. # Type Loss

Definition

Type loss means subclass-specific methods are not accessible.

Features

- Compile-time restriction
- Only superclass methods visible

Example

```
Animal a = new Dog();  
// a.fetch(); // compile-time error
```

Remark

- Type loss is **intentional and safe**
-

10.9. Cast

Definition

A cast converts a reference to another compatible type.

Features

- Uses (Type) syntax
- Checked at runtime

Example

```
Dog d = (Dog) a;
```

Remark

- Wrong cast causes `ClassCastException`
-

10.10. Summary (Exam Focus)

- Use inheritance to model **is-a**
- Subclass is a subtype
- Subtype substitution enables polymorphism
- Polymorphic variables cause type loss
- Use cast only when necessary and safe